

Architectural Tradeoffs in Low-Latency Algorithmic Trading with Predictive Signals

Abhinav Randive

March 4, 2026

1 Introduction

Financial markets have increasingly become dominated by automated trading systems that make decisions and execute trades at extremely high speeds. These systems rely on algorithmic strategies that analyze market data and react within milliseconds or even microseconds. As a result, the design of trading infrastructure has become just as important as the predictive models used to generate trading signals. Even when predictive models are accurate, delays in processing or order execution can eliminate any potential advantage in highly competitive markets.

Recent advances in machine learning have led to significant interest in applying predictive models to financial markets. Researchers have explored techniques ranging from classical statistical models to deep learning architectures for forecasting stock price movements. These models often attempt to identify patterns in historical price data, macroeconomic indicators, or market sentiment signals in order to generate short-term predictions about market behavior.

However, predictive accuracy alone does not guarantee profitability in financial markets. In practice, the ability to act on predictions depends on the latency of the trading system and the structure of the market itself. Modern electronic markets operate using limit order books and price-time priority mechanisms, meaning that orders arriving even slightly later than competitors may lose execution priority. This creates a critical interaction between prediction, system architecture, and market microstructure.

The goal of this project is to study these architectural tradeoffs by developing an event-driven trading simulation system. The system replays historical market data, processes events sequentially, generates short-term predictive signals, and measures the impact of latency and execution constraints on simulated trading outcomes. By focusing on the infrastructure that connects predictive models with market execution, this work aims to bridge the gap between machine learning research and practical trading system design.

2 Related Works

3 Methodology

This project begins by constructing the foundational infrastructure required for an event-driven trading system simulation. At this stage, the focus is on market data ingestion and deterministic event replay, which form the basis for analyzing architectural tradeoffs in low-latency trading systems.

3.1 Market Data Acquisition

Historical market data is downloaded using the Yahoo Finance API through the `yfinance` Python library. The current implementation retrieves recent S&P 500 index data at one-minute intervals and stores timestamped price and volume information in CSV format. The preprocessing step flattens multi-index columns, standardizes column names, and converts timestamps to integer Unix time for consistent processing.

This approach ensures reproducibility and establishes a structured input format for the simulation pipeline. Using historical replay rather than live feeds aligns with common practices in quantitative finance research, where controlled backtesting environments are used to evaluate trading logic [?, ?].

3.2 Event Abstraction

To support an event-driven architecture, market data is abstracted into discrete events. An `Event` data structure is defined with three fields: timestamp, event type, and payload. Event types are enumerated to allow future extension for order submission, cancellation, and execution events.

Each market update is represented as a structured event object, enabling uniform handling of data within the system. This abstraction follows principles of real-time stream processing systems, where events are processed sequentially and deterministically [?]. Event abstraction is critical for modeling realistic trading environments, where exchanges operate as event-driven systems governed by limit order book mechanics [?].

3.3 Deterministic Market Replay

A replay engine iterates sequentially over historical market data and converts each row into a `MARKET_UPDATE` event. The replay mechanism maintains an internal index and exposes methods to check for remaining events and retrieve the next event in chronological order.

Deterministic replay prevents look-ahead bias and ensures that events are processed strictly in historical sequence. This is essential when studying the interaction between prediction and execution, since improper ordering can artificially inflate model performance [?]. Although a full event dispatcher and execution engine have not yet been implemented, the replay system establishes the primary input stream for the future low-latency architecture.

4 Conclusion